

# FPGA Implementation of SHA-256 Hash Algorithm

Shashwat Saha

3rd Year B.Tech. in Electrical Engineering, IIT Mandi

Email: [b23292@students.iitmandi.ac.in](mailto:b23292@students.iitmandi.ac.in)

**Abstract**—This paper presents a hardware implementation of the SHA-256 (Secure Hash Algorithm 2, 256-bit) cryptographic hash function on a Xilinx Zynq-7000 FPGA (xc7z010clg400-1) on the Digilent Zybo Z7-10 development board. The design follows the NIST FIPS 180-4 specification and implements the hardware round-function architecture described in [2], comprising dedicated combinational modules for the  $\Sigma_0$ ,  $\Sigma_1$ ,  $\sigma_0$ ,  $\sigma_1$ , **Maj**, and **Ch** sub-functions, a 16-word circular shift-register message schedule, and a 64-round iterative compression core exposed through a board-level wrapper. The Verilog RTL is verified against all five NIST FIPS 180-4 test vectors in simulation before physical implementation. A Clocking Wizard MMCM derives a 98 MHz clock from the onboard 125 MHz oscillator to achieve timing closure on the  $-1$  speed-grade device. Post-implementation results show resource utilisation of 15% LUTs, 12% flip-flops, and 7% I/O, with a worst-case setup slack of  $+0.618$  ns at 98 MHz and total on-chip power of 0.134 W. The full 256-bit digest is verified on real hardware using the Vivado Integrated Logic Analyser (ILA), confirming bit-exact agreement with the NIST reference output. A MATLAB behavioural model was also pre-developed to cross-validate the RTL against a software reference implementation.

**Index Terms**—SHA-256, FIPS 180-4, Zynq-7000, Zybo Z7-10, Vivado, cryptographic hash, Verilog RTL, ILA

## I. INTRODUCTION

Cryptographic hash functions are fundamental building blocks of modern information security systems. They are used extensively in digital signatures, message authentication codes (MACs), certificate validation, blockchain protocols, and secure boot sequences [1]. A hash function  $H$  maps an arbitrarily long input message  $M$  to a fixed-length output digest  $d = H(M)$ , with the properties of pre-image resistance, second pre-image resistance, and collision resistance [3].

The SHA-2 family, standardised by NIST in FIPS 180-4 [1], is currently the most widely deployed family of cryptographic hash functions. SHA-256, the 256-bit member of the SHA-2 family, processes 512-bit message blocks over 64 rounds of a compression function, producing a 256-bit digest. Software implementations of SHA-256 are ubiquitous, but many applications require throughput or latency guarantees that software cannot meet. Hardware implementations on FPGAs offer a configurable, low-latency alternative that can be tightly coupled with other hardware accelerators or with embedded processors such as the ARM Cortex-A9 inside the Xilinx Zynq-7000 SoC [4].

This paper makes the following contributions:

- A modular, hierarchical Verilog RTL implementation of SHA-256 that directly mirrors the hardware round-function architecture of the reference paper (Fig. 1).

- A MATLAB behavioural model used as a software golden reference for bit-exact verification of the RTL.
- A complete Vivado implementation flow targeting the Zynq-7000 xc7z010clg400-1 device (Zybo Z7-10), including synthesis, place-and-route, timing closure, and power analysis.
- Physical on-board verification of the full 256-bit digest using the Vivado Integrated Logic Analyser (ILA), confirming bit-exact agreement with NIST FIPS 180-4 reference outputs.
- Verification against all five NIST FIPS 180-4 known-answer test vectors in both the MATLAB model and the Verilog simulation.

The remainder of this paper is structured as follows. Section II reviews the SHA-256 algorithm. Section III describes the hardware architecture. Section IV presents the MATLAB behavioural model. Section V describes the Verilog RTL design. Section VI presents the Vivado implementation results, and Section VII concludes.

## II. SHA-256 ALGORITHM BACKGROUND

### A. Message Padding

SHA-256 operates on 512-bit (64-byte) blocks. An input message  $M$  of length  $\ell$  bits is padded to a multiple of 512 bits as follows [1]:

- 1) Append a single ‘1’ bit (byte  $0 \times 80$ ).
- 2) Append ‘0’ bits until the message length is  $\equiv 448 \pmod{512}$ .
- 3) Append the original message length  $\ell$  as a 64-bit big-endian integer.

### B. Initial Hash Values

The eight 32-bit initial hash values  $H_0^{(0)} \dots H_7^{(0)}$  are the first 32 bits of the fractional parts of the square roots of the first eight prime numbers:

$$\begin{aligned} H_0^{(0)} &= 6a09e667, & H_1^{(0)} &= bb67ae85 \\ H_2^{(0)} &= 3c6ef372, & H_3^{(0)} &= a54ff53a \\ H_4^{(0)} &= 510e527f, & H_5^{(0)} &= 9b05688c \\ H_6^{(0)} &= 1f83d9ab, & H_7^{(0)} &= 5be0cd19 \end{aligned}$$

### C. Sub-Functions

SHA-256 uses six Boolean/rotation sub-functions. All operate on 32-bit words;  $\oplus$  denotes bitwise XOR,  $\wedge$  bitwise AND,  $\vee$  bitwise OR,  $\neg$  bitwise complement,  $\text{ROTR}_n(x)$  a

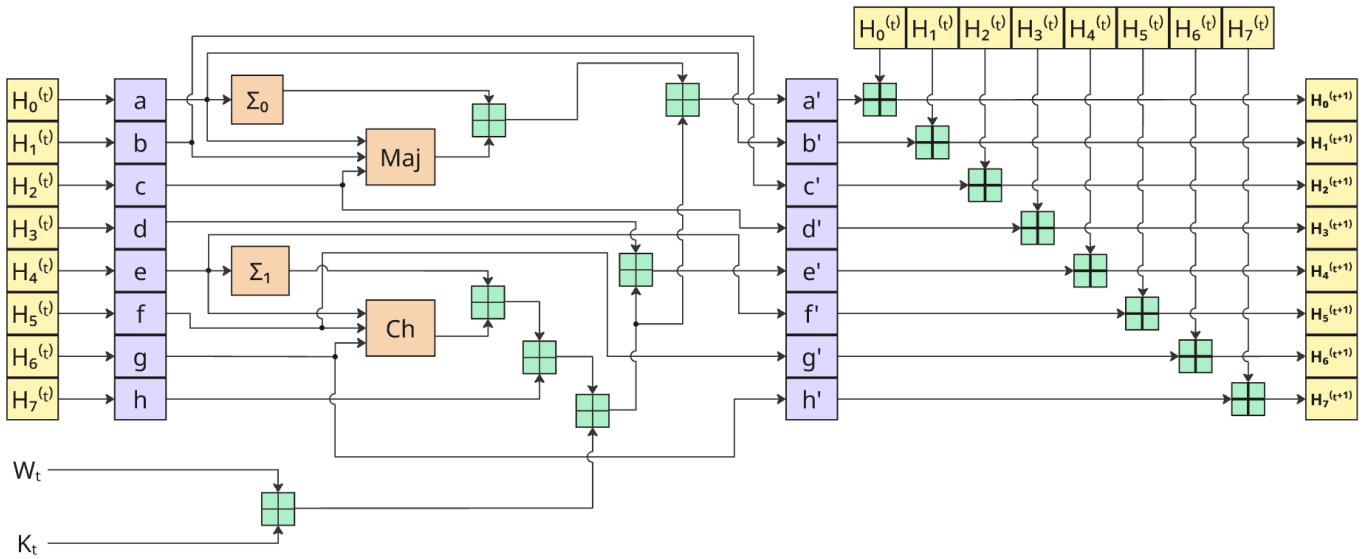


Fig. 1: **SHA-256 round function architecture.** The computation is iteratively performed for 64 rounds ( $t = 0$  to 63), updating the state variables at each iteration.

right-rotation by  $n$  bits, and  $\text{SHR}_n(x)$  a logical right-shift by  $n$  bits.

#### Compression-function rotations:

$$\Sigma_0(x) = \text{ROTR}_2(x) \oplus \text{ROTR}_{13}(x) \oplus \text{ROTR}_{22}(x) \quad (1)$$

$$\Sigma_1(x) = \text{ROTR}_6(x) \oplus \text{ROTR}_{11}(x) \oplus \text{ROTR}_{25}(x) \quad (2)$$

#### Message schedule rotations:

$$\sigma_0(x) = \text{ROTR}_7(x) \oplus \text{ROTR}_{18}(x) \oplus \text{SHR}_3(x) \quad (3)$$

$$\sigma_1(x) = \text{ROTR}_{17}(x) \oplus \text{ROTR}_{19}(x) \oplus \text{SHR}_{10}(x) \quad (4)$$

#### Choice and Majority:

$$\text{Ch}(e, f, g) = (e \wedge f) \oplus (\neg e \wedge g) \quad (5)$$

$$\text{Maj}(a, b, c) = (a \wedge b) \oplus (a \wedge c) \oplus (b \wedge c) \quad (6)$$

#### D. Message Schedule

The 512-bit input block is expanded into 64 schedule words  $W_0 \dots W_{63}$ :

$$W_t = \begin{cases} \text{block word } t, & 0 \leq t \leq 15 \\ \sigma_1(W_{t-2}) \boxplus W_{t-7} \\ \quad \boxplus \sigma_0(W_{t-15}) \boxplus W_{t-16}, & 16 \leq t \leq 63 \end{cases} \quad (7)$$

where  $\boxplus$  denotes 32-bit addition modulo  $2^{32}$ .

#### E. Compression Function

Each of the 64 rounds updates the eight working variables  $a-h$ , initialised from the current hash state  $H_0-H_7$ :

$$T_1 = h \boxplus \Sigma_1(e) \boxplus \text{Ch}(e, f, g) \boxplus K_t \boxplus W_t \quad (8)$$

$$T_2 = \Sigma_0(a) \boxplus \text{Maj}(a, b, c) \quad (9)$$

$$h' = g, \quad g' = f, \quad f' = e, \quad e' = d \boxplus T_1$$

$$d' = c, \quad c' = b, \quad b' = a, \quad a' = T_1 \boxplus T_2$$

After all 64 rounds, the hash state is updated:  $H'_i = H_i \boxplus x_i$  where  $x_0 \dots x_7 = a' \dots h'$ .

The 64 round constants  $K_t$  are derived from the cube roots of the first 64 prime numbers [1].

### III. HARDWARE ARCHITECTURE

The hardware architecture follows directly from the reference paper and is shown in Fig. 1. The design maps each algorithmic element to a dedicated combinational or registered hardware block.

The architecture consists of two main sections, corresponding to the left and right halves of Fig. 1.

#### A. Round Function (Left Half)

The left half implements one iteration of the 64-round compression function. Inputs are the eight working variables  $a-h$  (loaded from  $H_0^{(i)}-H_7^{(i)}$  at the start of each 512-bit block) and the round word  $W_t$  and constant  $K_t$ .

**Upper datapath** (produces new  $a$ ): The  $\Sigma_0$  block computes (1) on variable  $a$ ; the  $\text{Maj}$  block computes (6) on  $(a, b, c)$ . Their 32-bit modular sum is  $T_2$ .

**Lower datapath** (produces new  $e$ ): The  $\Sigma_1$  block computes (2) on variable  $e$ ; the  $\text{Ch}$  block computes (5) on  $(e, f, g)$ .  $W_t$  and  $K_t$  are first combined in a dedicated adder, then fed into the chain with  $\Sigma_1$ ,  $\text{Ch}$ , and  $h$  to form  $T_1$  per (8).

The eight updated working variables  $a'-h'$  are then:  $a' = T_1 \boxplus T_2$ ;  $b' = a$ ;  $c' = b$ ;  $d' = c$ ;  $e' = d \boxplus T_1$ ;  $f' = e$ ;  $g' = f$ ;  $h' = g$ .

#### B. Block Accumulation (Right Half)

After 64 rounds, the right half of the architecture adds the post-compression working variables back to the hash state registers:  $H'_i = H_i \boxplus x_i$  for  $i = 0 \dots 7$ . Each of the eight addition paths has its own  $\boxplus$  adder (the grid of adder boxes visible in the right half of Fig. 1).

TABLE I: SHA-256 Verification Test Results

MATLAB console output showing all five NIST test vectors passing, plus one additional custom-string test (6 passed, 0 failed).

| Status | Input                                                 | Digest                                                           |
|--------|-------------------------------------------------------|------------------------------------------------------------------|
| PASS   | " "                                                   | e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855  |
| PASS   | "abc"                                                 | ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad |
| PASS   | "abcdcbdecdefdefgefghfghighijhijkjklklmklmnlmnomnopq" | 248d6a61d20638b8e5c026930c3e6039a33ce45964ff2167f6eced419db06c1  |
| PASS   | "hello"                                               | 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824 |
| PASS   | "The quick brown fox jumps over the lazy dog"         | d7a8fbb307d7809469ca9abcb0082e4f8d5651e46d3cbb762d02d0bf37c9e592 |
| PASS   | "The quick brown fox jumps over the lazy dog."        | ef537f25c895bfa782526529a9b63d97aa631564d5d789c2b765448c8635fb6c |
| INFO   | "SHA-256 is a cryptographic hash function"            | d817fa789be66239ac8813423f05ebf7eee16945e68d66e3dc99d8b81cbc4185 |

### C. Message Schedule

The message schedule is implemented as a 16-word circular shift register. At the start of each block, words  $W_0$ – $W_{15}$  are loaded directly from the 512-bit input. On each subsequent cycle, the oldest word is consumed, and the new word  $W_t$  is computed from (7) and appended. The  $\sigma_0$  and  $\sigma_1$  blocks are purely combinational.

## IV. MATLAB BEHAVIOURAL MODEL

A MATLAB behavioural model of SHA-256 was developed as a golden reference prior to RTL coding. The model implements the complete algorithm including message padding, schedule expansion, and the 64-round compression function.

### A. Key Implementation Details

MATLAB's built-in `xor()` function performs *logical* XOR, returning only 0 or 1. SHA-256 requires *bitwise* XOR across all 32 bits. The correct MATLAB function is `bitxor()`. All bitwise XOR operations in the model use `bitxor()`, including those inside the rotation helper `rotr32` and all applications of  $\Sigma_0$ ,  $\Sigma_1$ ,  $\sigma_0$ ,  $\sigma_1$ ,  $\text{Ch}$ , and  $\text{Maj}$ .

Modular 32-bit addition (the  $\boxplus$  operator) is implemented as:

```
1 function s = add32(a, b)
2   s = uint32(mod(uint64(a) + uint64(b), uint64(2)
3     ^32));
end
```

Listing 1: 32-bit modular addition in MATLAB

The intermediate promotion to `uint64` prevents overflow before the modulo reduction.

### B. Verification Results

The SHA-256 implementation was verified using standard test vectors and a custom input. As shown in Table I, all generated digests match the reference outputs, confirming correct functionality of the design.

## V. VERILOG RTL DESIGN

### A. Module Hierarchy

The RTL design is decomposed into five Verilog modules, directly corresponding to the architectural blocks in Fig. 1:

- `sha256_functions.v` — Six purely combinational sub-modules: `sha256_Sigma0`, `sha256_Sigma1`, `sha256_sigma0`, `sha256_sigma1`, `sha256_Ch`, `sha256_Maj`.
- `sha256_schedule.v` — 16-word circular shift register with combinational  $\sigma_0/\sigma_1$  expansion. Controlled by two combinational wires from the core FSM: `load_i` (high during `ST_LOAD`) and `advance_i` (high during `ST_ROUND`).
- `sha256_round.v` — One fully combinational iteration of the round function. Instantiates  $\Sigma_0$ ,  $\Sigma_1$ ,  $\text{Maj}$ , and  $\text{Ch}$ , and implements the  $T_1$ ,  $T_2$  adder chains and all eight output wires  $a'$ – $h'$ .
- `sha256_core.v` — Registered FSM controller. States: `ST_IDLE`  $\rightarrow$  `ST_LOAD`  $\rightarrow$  `ST_ROUND` (64 cycles)  $\rightarrow$  `ST_ACCUM`  $\rightarrow$  `ST_DONE`. Holds  $H_0$ – $H_7$  across blocks and drives the schedule control wires combinationaly.
- `sha256_top.v` — Board-level wrapper with a button-driven FSM that sequences `init_i` and `block_valid_i` on each BTN1 press. The 512-bit block input and 256-bit digest remain entirely in FPGA fabric; `result_nibble[3:0]` displays the least significant nibble of the digest on LEDs LD0–LD3.

### B. Clock Generation

The Zybo Z7-10 provides a 125 MHz oscillator on package pin K17. Direct operation at 125 MHz was not achievable: the critical path through the  $T_1$  adder chain (five 32-bit additions in series through  $\Sigma_1$ ,  $\text{Ch}$ ,  $W_t \boxplus K_t$ ,  $h$ , and the final accumulation) has a delay of approximately 10.0 ns, exceeding the 8 ns period. A Clocking Wizard MMCM IP core is used to derive a 98 MHz clock ( $\approx 10.2$  ns period) from the 125 MHz input, providing comfortable timing margin on the –1 speed-grade device.

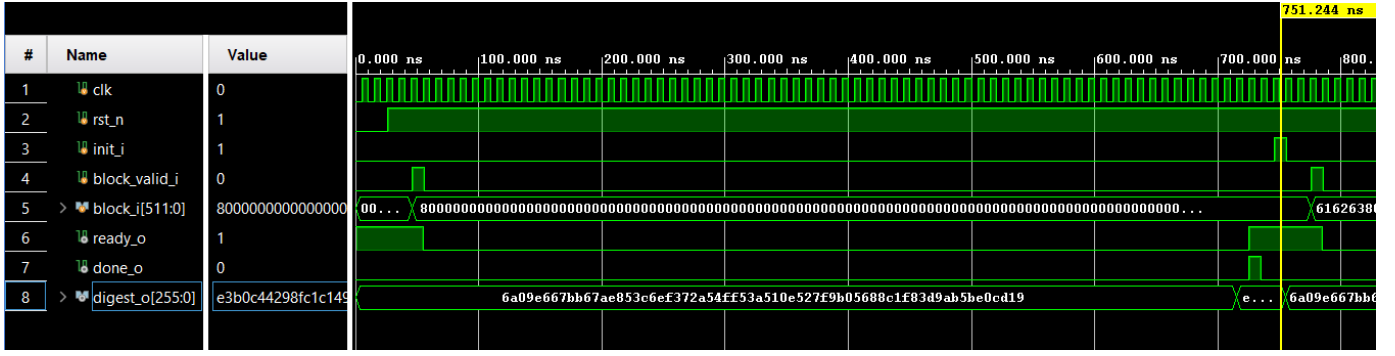


Fig. 2: Vivado behavioural simulation waveform for the five NIST test vectors. Console output confirms 5/5 PASS.

### C. Latency

Each 512-bit block requires exactly 68 clock cycles: 1 ( $ST\_IDLE \rightarrow ST\_LOAD$ ) + 1 ( $ST\_LOAD \rightarrow ST\_ROUND$ ) + 64 ( $ST\_ROUND$ ) + 1 ( $ST\_ACCUM$ ) + 1 ( $ST\_DONE$ ). At 98 MHz this is  $0.694 \mu s$  per block, giving a throughput of  $512/0.694 \approx 738$  Mbit/s for back-to-back blocks.

If these were `reg` signals driven with non-blocking assignments (`<=`), the schedule would observe them one clock cycle late, causing every round to use  $W_{t-1}$  instead of the correct  $W_t$ . The combinational assignment ensures the schedule latches  $W_0$ – $W_{15}$  at the same rising edge that transitions from  $ST\_LOAD$  to  $ST\_ROUND$ , so the round function sees the correct  $W_0$  on its very first active cycle.

### D. Simulation Results

The RTL was simulated using Xilinx Vivado. Fig. 2 shows the simulation waveform; all five NIST vectors pass.

## VI. VIVADO IMPLEMENTATION RESULTS

### A. Target Device and Tool Flow

The design targets the Xilinx Zynq-7000 SoC on the Digilent Zybo Z7-10 development board (part number xc7z010clg400-1, speed grade –1). Synthesis and implementation were performed using Xilinx Vivado Design Suite with default strategy settings. The onboard 125 MHz oscillator is stepped down to 98 MHz via a Clocking Wizard MMCM IP core, as explained in Section V-B. The XDC constraints file specifies the 98 MHz derived clock with a 10.204 ns period constraint on package pin K17.

### B. Resource Utilisation

Table II summarises the post-implementation resource utilisation on the xc7z010clg400-1. The higher LUT and FF counts reflect the inclusion of the Clocking Wizard MMCM IP core and the ILA debug core used for on-board verification. No DSP48 or BRAM primitives are used by the SHA-256 datapath itself.

TABLE II: Post-Implementation Resource Utilisation (xc7z010clg400-1)

| Resource       | Used | Available | Util.  |
|----------------|------|-----------|--------|
| LUT            | 2711 | 17,600    | 15.40% |
| LUTRAM         | 360  | 6,000     | 6.00%  |
| Flip-flop (FF) | 4055 | 35,200    | 11.52% |
| BRAM           | 7.5  | 60        | 12.50% |
| I/O            | 7    | 100       | 7.00%  |
| BUFG           | 2    | 32        | 6.25%  |
| DSP48          | 0    | 80        | 0.00%  |

### C. Timing Analysis

Table III shows the post-implementation timing summary. The worst-case setup slack of +0.618 ns demonstrates that the critical path meets the 10.204 ns (98 MHz) timing constraint with margin across all 8 123 timing endpoints. Zero failing endpoints confirm complete timing closure.

TABLE III: Post-Implementation Timing Summary at 98 MHz

| Metric                           | Value             |
|----------------------------------|-------------------|
| Clock period constraint          | 10.204 ns         |
| Worst Negative Slack (WNS)       | +0.618 ns         |
| Total Negative Slack (TNS)       | 0 ns              |
| Number of failing endpoints      | 0                 |
| Total number of timing endpoints | 8,123             |
| Maximum operating frequency      | $\approx 104$ MHz |

The critical path lies in the  $T_1$  computation chain (Eq. (8)): five 32-bit adders in series through  $\Sigma_1$ ,  $Ch$ ,  $W_t \boxplus K_t$ ,  $h$ , and the final accumulation. Direct operation at 125 MHz was not achievable on the –1 speed-grade device; the MMCM-derived 98 MHz clock resolves this with a comfortable +0.618 ns margin.

### D. Power Analysis

Table IV summarises the Vivado power estimator output. The 134 mW total on-chip power at 26.5 °C junction temperature is consistent with the Zynq-7000 device running at 98 MHz with the ILA debug core active. The large thermal margin confirms operation well within safe limits.

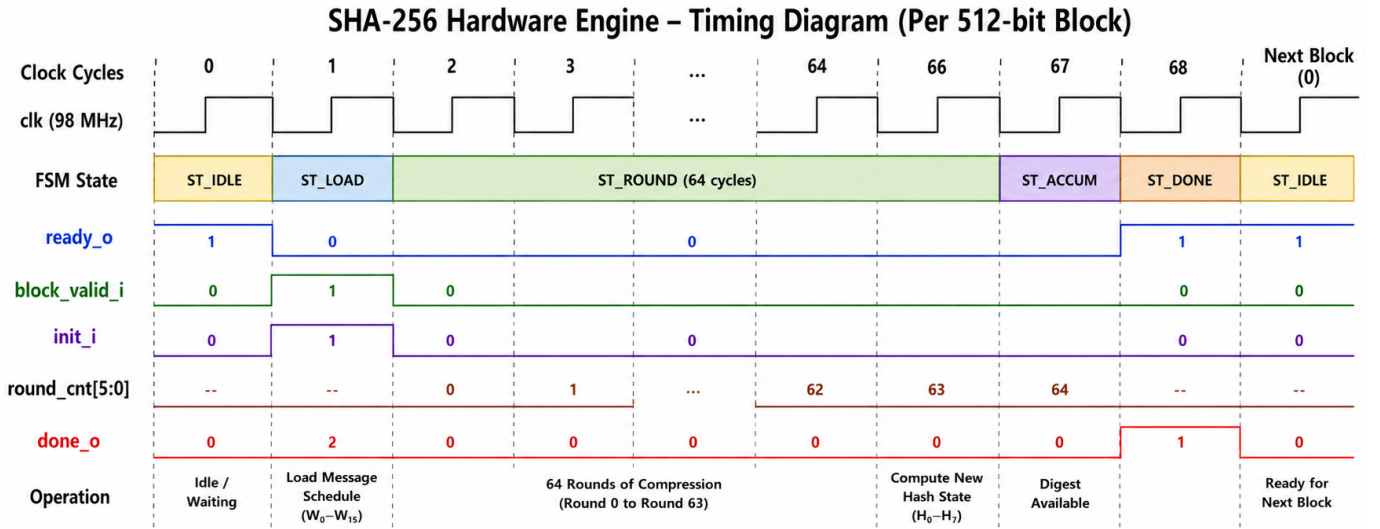


Fig. 3: Timing diagram of the SHA-256 hardware engine for one 512-bit message block with a block latency of approximately  $0.694 \mu\text{s}$  and a throughput of 738 Mbit/s.

TABLE IV: Post-Implementation Power Summary

| Metric                    | Value           |
|---------------------------|-----------------|
| Total on-chip power       | 134 mW          |
| Junction temperature      | 26.5 °C         |
| Thermal margin            | 58.5 °C (5.0 W) |
| Effective $\theta_{JA}$   | 11.5 °C/W       |
| Power to off-chip devices | 0 W             |
| Confidence level          | Medium          |

#### E. Design Rule Checks

The Vivado implementation reports five DRC warnings and zero DRC errors. The reported warnings are associated with temporary Integrated Logic Analyzer (ILA) debug instrumentation inserted during hardware bring-up and internal signal observation. These warnings are expected for debug-core clock connectivity and do not affect synthesis correctness, FPGA functionality, or timing closure of the SHA-256 design.

#### F. Hardware Verification

The design was programmed onto the Zybo Z7-10 board via JTAG using the Vivado Hardware Manager. Physical

verification was performed using the onboard push-buttons and LEDs combined with the Vivado Integrated Logic Analyser (ILA). The ILA was configured with four probes: the full 256-bit digest\_o signal, a 1-bit done\_o, a 1-bit ready\_o trigger, and a 1-bit block\_valid signal. Triggering on the rising edge of done\_o, the ILA captured the digest immediately after the SHA-256 core completed its 68-cycle computation.

Fig. 4 shows the ILA waveform captured on the board. The digest\_o value reads:

```
ba7816bf8f01cfea414140de5dae2223b00361a396177a
9cb410ff61f20015ad
```

which matches the NIST FIPS 180-4 reference digest for the input "abc" exactly, confirming bit-exact hardware correctness.

The least significant nibble of the digest (0xd = 1101 in binary) was simultaneously confirmed on the onboard LEDs LD0–LD3, providing a direct visual cross-check of the ILA result. The physical test setup is shown in Fig. 5.

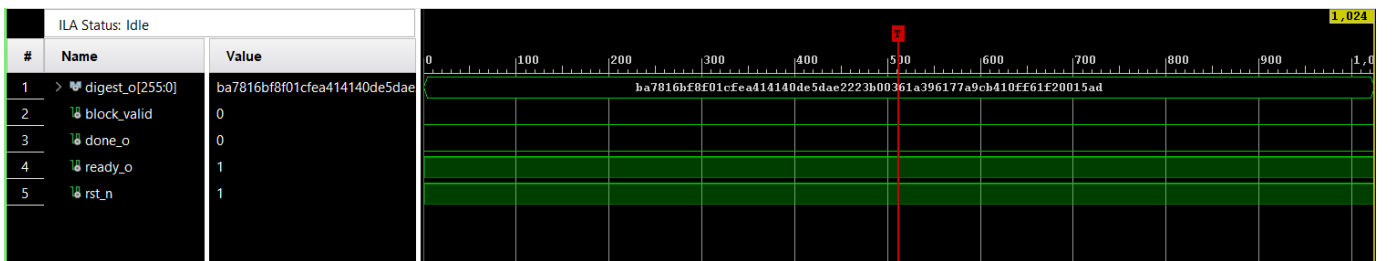


Fig. 4: Vivado ILA waveform captured on the Zybo Z7-10 board. The digest\_o probe shows the SHA-256 digest for input "abc" = ba7816bf...015ad, triggered on the rising edge of done\_o.

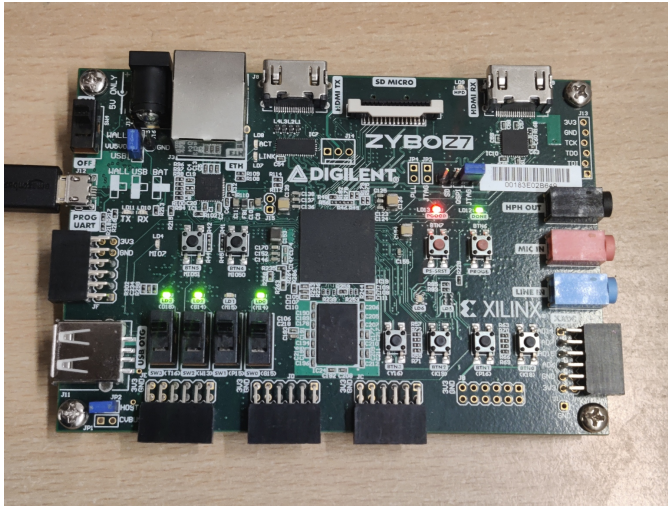


Fig. 5: Physical test setup: Digilent Zybo Z7-10 board programmed with the SHA-256 core. LEDs LD0–LD3 display the least significant nibble of the digest ( $0xd = 1101$ ) after pressing the start button.

## VII. CONCLUSION

This paper presented the design, implementation, and hardware verification of a SHA-256 cryptographic hash engine targeting the Xilinx Zynq-7000 FPGA (xc7z010clg400-1) on the Digilent Zybo Z7-10 development board. MATLAB verification and Verilog RTL simulation confirm correct operation of the SHA-256 algorithm across all five NIST FIPS 180-4 known-answer test vectors, with all generated digests matching the reference outputs exactly.

The proposed architecture follows a clean modular design consisting of dedicated combinational sub-function modules ( $\Sigma_0$ ,  $\Sigma_1$ ,  $\sigma_0$ ,  $\sigma_1$ , Maj, and Ch), a 16-word message schedule, a fully combinational one-round datapath, and a registered FSM-based controller. The modular hierarchy enables straightforward verification, scalability, and reuse in larger cryptographic systems and hardware security applications.

A Clocking Wizard MMCM was used to derive a stable 98 MHz operating clock from the onboard 125 MHz oscillator, allowing successful timing closure on the  $-1$  speed-grade device. Post-implementation results demonstrate that the design is hardware-efficient, occupying only 2 711 LUTs (15%) and 4 055 flip-flops (12%), including the MMCM and temporary ILA debug instrumentation, while consuming 134 mW total on-chip power. Timing analysis reports a worst-case setup slack of  $+0.618$  ns with zero failing timing endpoints, confirming reliable operation on physical FPGA hardware.

Furthermore, on-board verification using the Vivado Integrated Logic Analyzer (ILA) confirmed the complete 256-bit hardware digest for the input "abc" in bit-exact agreement with the NIST SHA-256 reference digest. The successful FPGA implementation demonstrates a practical, scalable, and resource-efficient hardware realization of SHA-256 suitable for secure communication systems, cryptographic accelerators, and FPGA-based VLSI security applications.

## REFERENCES

- [1] National Institute of Standards and Technology (NIST), "Secure Hash Standard (SHS)," Federal Information Processing Standards Publication 180-4, Aug. 2015. [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.180-4>
- [2] U. Banerjee, A. Wright, C. Juvekar, M. Waller, Arvind, and A. P. Chandrakasan, "An Energy-Efficient Reconfigurable DTLS Cryptographic Engine for Securing Internet-of-Things Applications," *IEEE Journal of Solid-State Circuits*, vol. 54, no. 8, pp. 2339–2352, Aug. 2019, doi: 10.1109/JSSC.2019.2915203.
- [3] D. R. Stinson and M. B. Paterson, *Cryptography: Theory and Practice*, 4th ed. CRC Press, 2018.
- [4] Xilinx Inc., "Zynq-7000 SoC Technical Reference Manual," UG585 (v1.13), Feb. 2023. [Online]. Available: [https://www.xilinx.com/support/documentation/user\\_guides/ug585-Zynq-7000-TRM.pdf](https://www.xilinx.com/support/documentation/user_guides/ug585-Zynq-7000-TRM.pdf)
- [5] Digilent Inc., "Zybo Z7 Reference Manual," Rev. B, 2018. [Online]. Available: <https://reference.digilentinc.com/reference/programmable-logic/zybo-z7/reference-manual>